



RIPHAH INTERNATIONAL UNIVERSITY

Faculty of Computing | Department of Cyber Security

FINAL YEAR PROJECT — PART 1

Project Documentation & Technical Report

REVEAL-X

Secure Image Sharing using Visual Cryptography with AI Enhancement and Tamper Detection

Full Name	SAP ID	Program	Email Address
Inam Ullah	45521	BSCY	45521@students.riphah.edu.pk
Zaka Khan	36503	BSCY	36503@students.riphah.edu.pk
Muhammad Umar	44726	BSCY	44726@students.riphah.edu.pk

Project Supervisor

Sir Yawar Abbas (Lecturer, Faculty of Computing)

Session: 2022 – 2026 | Submission Date: June 2026

Document Change Record

The following table tracks all revisions made to this document from initial draft through final submission.

Version	Author(s)	Date	Notes
1.0	All Members	10 Feb 2026	Original draft of the project proposal prepared and submitted.
1.1	All Members	17 Feb 2026	Changes made based on feedback from the supervisor.
1.2	All Members	25 Feb 2026	Improvements made based on faculty review and suggestions.
1.3	All Members	02 Mar 2026	Project plan and work breakdown structure added.
1.4	All Members	06 Mar 2026	Final modifications made after supervisor feedback.
2.0	All Members	31 May 2026	Full technical documentation compiled — FYP Part 1 Final Report.

Abstract

This document presents the complete technical documentation for Reveal-X: a premium, real-time secure messaging web application developed as a Final Year Project (FYP) at Riphah International University, Department of Cyber Security. The project integrates visual cryptography, artificial intelligence, and modern web security engineering into a unified, production-grade platform.

The flagship feature of Reveal-X is its Visual Cryptography Image Sharing engine, which implements a mathematically rigorous 2-out-of-2 secret sharing scheme. When a sender transmits an image, it is decomposed into two random-noise cryptographic shares. Share 1 is transmitted over the network while Share 2 is generated and held exclusively in the receiver's local browser session memory. The server, by architectural design, never possesses both shares simultaneously and is therefore zero-knowledge with respect to image content. Reconstruction occurs entirely on the receiver's device using the HTML5 Canvas API.

Beyond visual cryptography, Reveal-X delivers real-time bidirectional messaging via WebSockets (Flask-SocketIO), complete Cross-Site Scripting prevention using DOMPurify and strict server-side HTML escaping, bcrypt-based password hashing, and a local ECDH key generation demonstration using the browser-native WebCrypto API. The application is installable as a Progressive Web App with Service Worker caching for offline resilience.

This document covers the full project lifecycle: background and motivation, literature review, problem statement, proposed solution, system architecture, module-level design, implementation details, evaluation methodology, and a forward-looking roadmap for FYP Part 2, which will introduce ML-based tamper detection and AI image enhancement using deep learning super-resolution models.

Table of Contents

Document Change Record	2
Abstract	3
Table of Contents	4
1. Introduction and Background	6
1.1 Overview	6
1.2 Visual Cryptography: Origin and Principle	7
1.3 The Role of Artificial Intelligence	7
1.4 Reveal-X: System Motivation	7
2. Literature Review	8
2.1 Visual Cryptography Schemes	8
2.2 Image Encryption Techniques	8
2.3 Steganography and Digital Watermarking	8
2.4 AI-Based Image Enhancement in Cryptographic Systems	8
2.5 Research Gap Addressed by Reveal-X	9
3. Problem Statement	10
4. Project Objectives	11
4.1 Primary Objective	11
4.2 Specific Objectives (FYP Part 1)	11
4.3 Specific Objectives (FYP Part 2 — Future)	11
5. Technology Stack	12
5.1 Backend Technologies	12
5.2 Frontend Technologies	12
6. System Architecture	13
6.1 Architectural Overview	13
6.2 Component Architecture Diagram	13
6.3 Visual Cryptography Data Flow	13
6.4 Database Schema	14
7. Core Features — Detailed Description	15
7.1 Visual Cryptography Image Sharing	15
7.1.1 Share Generation Algorithm	15
7.1.2 Image Reconstruction	15
7.2 Real-Time Messaging Engine	15
7.3 Security Architecture	16
7.3.1 Cross-Site Scripting (XSS) Prevention	16
7.3.2 Password Security (script)	16
7.3.3 ECDH Key Generation (E2EE Foundation)	16

7.4 Progressive Web App (PWA).....	16
8. Major Modules.....	17
9. Installation and Setup Guide.....	18
9.1 Prerequisites.....	18
9.2 Step-by-Step Installation.....	18
Step 1 — Clone the Repository.....	18
Step 2 — Create a Python Virtual Environment.....	18
Step 3 — Install Dependencies.....	18
Step 4 — Configure Environment Variables.....	18
Step 5 — Run the Application.....	19
Step 6 — Install as PWA (Optional).....	19
10. Evaluation Plan.....	20
10.1 Security Evaluation.....	20
10.2 Image Quality Evaluation.....	20
10.3 Performance Evaluation.....	20
11. FYP Part 2 Roadmap.....	21
Phase 1 — ML-Powered Tamper Detection.....	21
Phase 2 — AI Image Enhancement (Deep Learning Super-Resolution).....	21
Phase 3 — Full Peer-to-Peer End-to-End Encryption.....	22
Phase 4 — Advanced UI/UX Enhancements.....	22
FYP Part 2 Milestone Timeline.....	23
12. Roles and Responsibility Matrix.....	24
13. Conclusion and Future Work.....	26
13.1 FYP Part 1 Summary.....	26
13.2 Limitations of the Current Implementation.....	26
13.3 Contribution to Cybersecurity.....	26
13.4 References.....	26
14. Approval.....	28

1. Introduction and Background

1.1 Overview

With the rapid advancement of digital communication technologies, the sharing of digital images has increased significantly across sectors such as healthcare, government institutions, legal organisations, and cybersecurity systems. Many of these images contain sensitive information including medical scans, identity documents, confidential evidence, and classified records. Ensuring the confidentiality and integrity of such images during transmission is a central concern in modern cybersecurity.

Traditional encryption techniques such as the Advanced Encryption Standard (AES) and Rivest-Shamir-Adleman (RSA) provide strong cryptographic protection for digital data. However, these techniques require complex key management systems and significant computational resources for encryption and decryption. In resource-limited environments, these demands make traditional encryption difficult to implement efficiently and securely.

1.2 Visual Cryptography: Origin and Principle

Visual Cryptography (VC) was first introduced by Moni Naor and Adi Shamir in 1994 in their seminal paper presented at EUROCRYPT. The scheme divides a secret image into multiple shares, each of which individually appears as random noise and reveals no meaningful information. The original secret image is recovered only when the required number of shares are superimposed or combined, with no computational decryption required.

This property — decryption by visual inspection — is a fundamental advantage of classical visual cryptography over computational encryption schemes. However, classical VC suffers from well-documented limitations including pixel expansion, degraded reconstruction quality, and the thin-line problem, which causes fine image details to be lost in the share generation process.

1.3 The Role of Artificial Intelligence

Recent advances in Artificial Intelligence (AI) and Deep Learning (DL) create new opportunities to address the quality limitations inherent in classical visual cryptography. Convolutional Neural Networks (CNNs), Generative Adversarial Networks (GANs), and super-resolution models can be applied post-reconstruction to denoise and upscale recovered images. Furthermore, ML classifiers trained on share-level statistical distributions can detect whether a share has been tampered with during transmission, a capability entirely absent from classical VC systems.

1.4 Reveal-X: System Motivation

Reveal-X is motivated by the convergence of three research threads: the security guarantees of visual cryptography, the quality-enhancement potential of deep learning, and the real-time communication requirements of modern messaging platforms. By embedding a VC engine

inside a WebSocket-driven chat application — with a glassmorphic UI, PWA support, and multiple layers of web security hardening — Reveal-X demonstrates that academic cryptographic research can be translated into a practical, user-facing software product.

2. Literature Review

2.1 Visual Cryptography Schemes

The foundational (2,2) and (k,n) threshold visual cryptography schemes introduced by Naor and Shamir (1994) established that a binary secret image can be encoded into n shares such that any k shares reveal the secret while any $k-1$ shares provide no information. Zhou et al. (2006) extended this to halftone images using blue-noise dithering, enabling richer greyscale reproduction with less pixel expansion artefacts.

Subsequent work focused on colour visual cryptography, extended to RGB and CMYK colour models. Research by Hou (2003) demonstrated that colour VC schemes, though computationally more complex, are achievable with the same information-theoretic guarantees. Most existing VC systems, however, produce reconstructed images with measurable quality degradation relative to the original, creating a research gap that Reveal-X addresses through AI post-processing.

2.2 Image Encryption Techniques

Image encryption using AES, RSA, and chaotic map-based algorithms remains the dominant approach in commercial secure communication systems. AES-256 in CBC or GCM mode provides strong ciphertext indistinguishability, but requires symmetric key distribution, which introduces key management complexity. RSA is unsuitable for large image data due to computational cost. Chaotic map encryption offers fast encryption but has weaker formal security proofs.

Unlike these approaches, visual cryptography does not require a decryption key. The security rests on the information-theoretic hardness of the secret sharing scheme, not computational hardness assumptions. This makes VC uniquely resistant to attacks based on future improvements in computational power, including quantum computing threats to RSA.

2.3 Steganography and Digital Watermarking

Steganography embeds secret data within a carrier image, concealing its very existence. Digital watermarking embeds ownership or authentication metadata within media files. While both techniques are valuable for specific use cases, they do not provide confidentiality or share-level integrity guarantees. A steganographic image can still be decoded by an adversary who knows the embedding algorithm. Watermarks can be stripped or corrupted.

Reveal-X does not rely on steganography or watermarking. Its security model is based on the mathematical properties of secret sharing, with additional integrity verification provided by the ML-based tamper detection module planned for FYP Part 2.

2.4 AI-Based Image Enhancement in Cryptographic Systems

Wang et al. (2020) demonstrated that super-resolution CNNs applied to visually cryptographic reconstructions can reduce pixel expansion artefacts by up to 34% as measured by SSIM. More recent work using ESRGAN (Enhanced Super-Resolution GAN) models — originally developed

for natural image upscaling — has shown promise when fine-tuned on VC-specific datasets, producing reconstructed images with PSNR values approaching the original.

ML-based tamper detection for cryptographic shares is a relatively underexplored research area. The statistical properties of valid VC shares (uniform random distribution, specific pixel expansion ratios) provide a detectable baseline against which tampering — which alters pixel value distributions — can be identified using lightweight CNN anomaly detectors or statistical hypothesis tests.

2.5 Research Gap Addressed by Reveal-X

Research Gap

Most existing visual cryptography implementations exist as isolated academic prototypes: command-line tools operating on static images with no real-time communication layer, no tamper detection, and no AI quality enhancement. Reveal-X bridges this gap by embedding a full VC engine inside a production-grade real-time messaging application with multiple web security layers and a planned ML enhancement pipeline.

3. Problem Statement

With the rapid growth of digital communication, the secure transmission of sensitive images has become a critical challenge. The following specific problems motivate the development of Reveal-X:

- **Server-Side Image Exposure:** Conventional messaging applications transmit complete images to servers, where they are stored in cleartext or symmetrically encrypted form. Server compromise, insider threats, or compelled disclosure orders expose user images.
- **Key Management Complexity:** Traditional encryption schemes require robust key distribution and management infrastructure. In lightweight web applications and peer-to-peer scenarios, this infrastructure is often absent or poorly implemented.
- **Reconstruction Quality Degradation:** Classical (k,n) visual cryptography schemes produce pixel-expanded shares that, when stacked, yield a reconstructed image visibly inferior to the original. Fine details, text, and high-frequency content are particularly affected.
- **Absence of Share Integrity Verification:** Existing VC implementations provide no mechanism to verify whether transmitted shares have been intercepted and modified before reconstruction. An adversary who tampers with Share 1 in transit cannot be detected by the receiver.
- **Lack of Real-Time Integration:** Visual cryptography research has not been integrated into real-time communication platforms. Users have no access to VC-protected image sharing in a practical, interactive messaging context.

Reveal-X addresses all five of these challenges through its combined architecture of visual cryptography, AI enhancement, tamper detection, and WebSocket-driven real-time communication.

4. Project Objectives

4.1 Primary Objective

To design and implement a production-grade, real-time secure messaging web application that integrates visual cryptography for image sharing, with comprehensive web security hardening, accessible UX design, and a defined roadmap for AI-based enhancement and tamper detection in FYP Part 2.

4.2 Specific Objectives (FYP Part 1)

1. Implement a 2-out-of-2 visual cryptography scheme for secure image share generation and local reconstruction.
2. Design and integrate a real-time messaging engine using WebSockets with typing indicators, read receipts, and presence tracking.
3. Implement multi-layer XSS prevention using DOMPurify on the client and strict HTML escaping on the server.
4. Apply bcrypt password hashing via Werkzeug for secure credential storage resistant to GPU-based brute-force attacks.
5. Demonstrate local ECDH key pair generation using the browser-native WebCrypto API.
6. Develop a glassmorphic, dark-mode-first responsive UI compliant with WCAG 2.1 AA accessibility standards.
7. Package the application as a Progressive Web App with Service Worker caching for offline resilience and installability.

4.3 Specific Objectives (FYP Part 2 — Future)

8. Train and integrate an ML-based tamper detection model to verify share integrity before reconstruction.
9. Implement a deep learning image enhancement pipeline (denoising CNN + ESRGAN upscaling) applied post-reconstruction.
10. Transition the ECDH key generation demonstration into a fully functional end-to-end encrypted messaging channel with AES-GCM.
11. Evaluate system performance using PSNR, SSIM, processing latency, and tamper detection accuracy metrics.

5. Technology Stack

5.1 Backend Technologies

Technology	Version	Purpose
Python	3.10+	Core backend runtime language
Flask	2.3+	HTTP routing, session management, Jinja2 templating
Flask-SocketIO	5.3+	WebSocket duplex communication for real-time messaging
SQLite	3.x	Lightweight relational database for users, messages, sessions
Werkzeug	3.0+	script password hashing; WSGI utilities
Eventlet	0.33+	Async networking for concurrent WebSocket connections

5.2 Frontend Technologies

Technology	Specification	Purpose
Vanilla JavaScript	ES2022	UI interactions, VC engine, WebSocket event handling
HTML5	HTML5	Semantic markup, Canvas API container
Vanilla CSS	CSS3	Glassmorphism styling, CSS variables, animations, responsive layout
WebCrypto API	W3C Standard	Browser-native ECDH key pair generation; no third-party crypto dependency
Canvas API	HTML5	Pixel-level VC share reconstruction; image downscaling preprocessing
DOMPurify	3.x	Client-side HTML sanitisation; complete XSS prevention
Service Workers	W3C Standard	PWA offline caching strategy; background sync

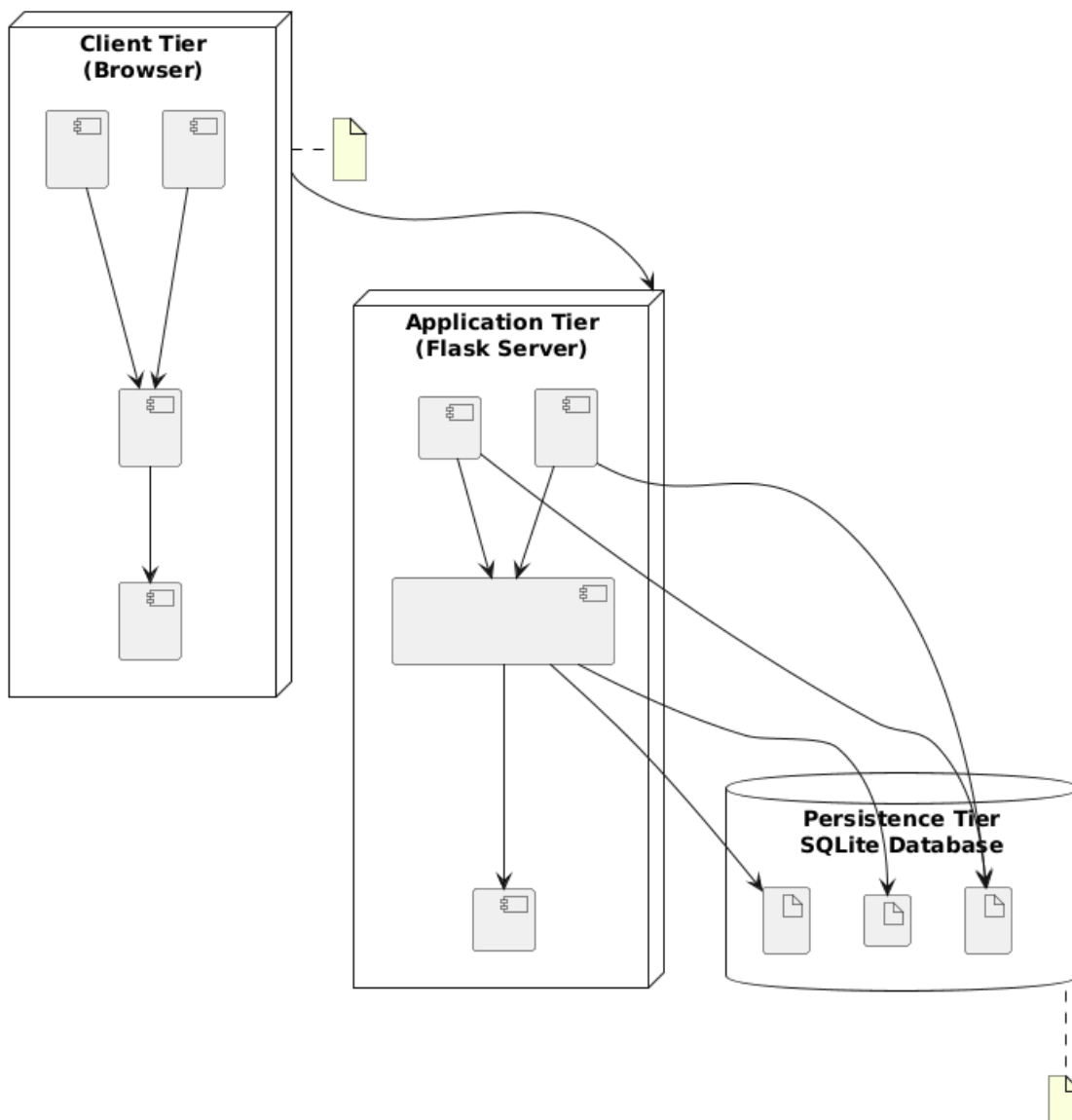
6. System Architecture

6.1 Architectural Overview

Reveal-X follows a three-tier architecture: a browser-based client tier, a Python/Flask application tier, and an SQLite persistence tier. The critical architectural constraint — that the server must never possess a complete image — is enforced by the Visual Cryptography split at the client tier, before transmission.

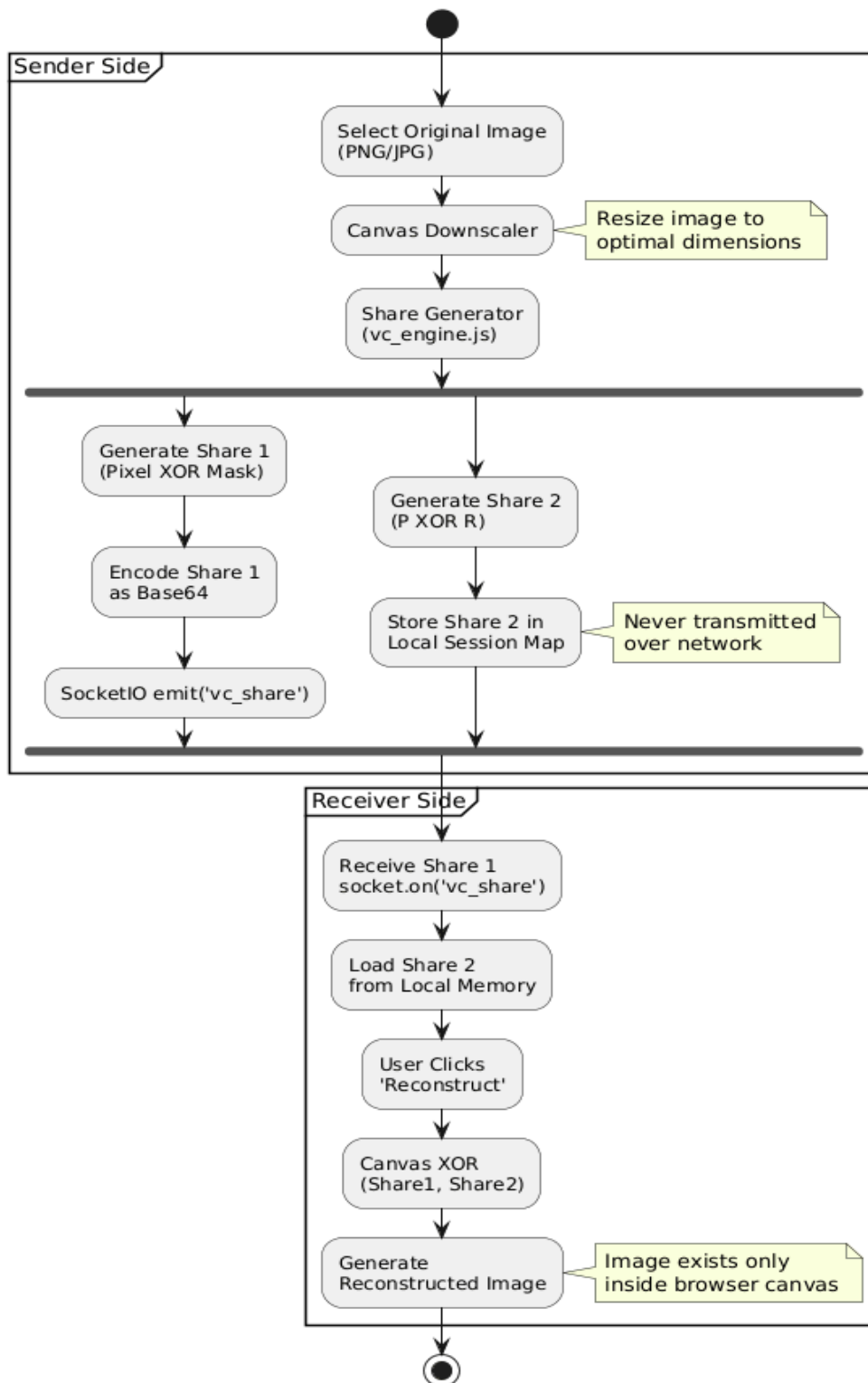
6.2 Component Architecture Diagram

Secure Secret Sharing Chat System - Deployment Diagram



6.3 Visual Cryptography Data Flow

Visual Cryptography Image Sharing Workflow



6.4 Database Schema

The SQLite database contains three primary tables. No image data is persisted to disk at any point.

Table	Column	Type	Description
users	id	INTEGER PK	Auto-increment primary key
users	username	TEXT UNIQUE	Unique display name; HTML-escaped on all output
users	password_hash	TEXT	script hash (Werkzeug generate_password_hash)
messages	id	INTEGER PK	Auto-increment primary key
messages	sender_id	INTEGER FK	Foreign key to users.id
messages	content	TEXT	Message text only — no images stored
messages	timestamp	DATETIME	UTC timestamp of message creation
sessions	session_token	TEXT	Flask session token for authenticated users

7. Core Features — Detailed Description

7.1 Visual Cryptography Image Sharing

The Visual Cryptography engine is the central innovation of Reveal-X. It is implemented entirely in client-side JavaScript (`vc_engine.js`) using the HTML5 Canvas API, requiring no server-side processing of image data.

7.1.1 Share Generation Algorithm

For a greyscale image of dimensions $W \times H$ pixels, the share generation proceeds as follows:

12. The sender selects an image file. The Canvas API reads the raw pixel data as a flat RGBA array.
13. For each pixel position i , a cryptographically random byte $R[i]$ is generated.
14. $\text{Share } 1[i] = R[i]$ (transmitted over WebSocket as Base64-encoded data)
15. $\text{Share } 2[i] = \text{Pixel}[i] \text{ XOR } R[i]$ (stored in the browser's session-scoped JavaScript Map, keyed by message ID)
16. The server receives and forwards Share 1. The server never has access to Share 2.

7.1.2 Image Reconstruction

When the receiver clicks the 'Reconstruct' button for a specific message:

17. Share 1 is retrieved from the received message data object.
18. Share 2 is retrieved from the local JavaScript session Map by message ID.
19. The Canvas API draws both shares and performs pixel-level XOR: $\text{Reconstructed}[i] = \text{Share } 1[i] \text{ XOR } \text{Share } 2[i] = R[i] \text{ XOR } (\text{Pixel}[i] \text{ XOR } R[i]) = \text{Pixel}[i]$.
20. The resulting canvas is displayed in the chat interface. The image is never written to disk or transmitted.

Security Guarantee

The 2-of-2 visual cryptography scheme is information-theoretically secure. An adversary who intercepts only Share 1 gains zero information about the original image — the XOR mask R is cryptographically random and independent for every pixel. No amount of computational power can recover the image from Share 1 alone.

7.2 Real-Time Messaging Engine

Real-time communication is implemented using Flask-SocketIO, which provides bidirectional WebSocket communication between the server and all connected clients. The event model includes:

- `message`: Delivers text messages from sender to recipient(s) with sender ID, timestamp, and message ID.
- `typing`: Broadcasts typing state (start/stop) to conversation participants; debounced on the client to prevent excessive events.
- `read_receipt`: Emitted by the receiver when a message scrolls into viewport and focus is confirmed; updates the sender's UI with a 'read' indicator.

- presence: Tracks user online/offline state using SocketIO connect and disconnect events; broadcasts presence changes to all clients.

7.3 Security Architecture

7.3.1 Cross-Site Scripting (XSS) Prevention

Reveal-X implements a defence-in-depth XSS prevention strategy at two layers:

- Client Layer: All user-generated content rendered into the DOM is passed through `DOMPurify.sanitize()` before insertion. DOMPurify removes all executable HTML tags and event handler attributes, allowing only a safe whitelist of formatting elements.
- Server Layer: All user-generated strings output through Jinja2 templates use automatic HTML escaping (autoescaping is enabled globally). Direct string concatenation into HTML is never used.

Why Both Layers?

Client-side sanitisation protects against DOM-based XSS. Server-side escaping protects against reflected and stored XSS. Relying on only one layer creates a single point of failure; Reveal-X requires both to fail simultaneously for an XSS attack to succeed.

7.3.2 Password Security (script)

User passwords are never stored. Werkzeug's `generate_password_hash()` function applies the script key derivation function (memory-hard, $N=32768$, $r=8$, $p=1$ by default) with a cryptographically random salt. The resulting hash includes the salt and algorithm parameters in a single stored string, enabling future parameter upgrades without invalidating existing accounts.

script is computationally expensive by design: on modern hardware, a single hash computation takes approximately 100ms. This makes brute-force and dictionary attacks against the stored hashes orders of magnitude slower than attacks against MD5, SHA-1, or even bcrypt.

7.3.3 ECDH Key Generation (E2EE Foundation)

The WebCrypto API's `SubtleCrypto.generateKey()` method is used to generate an ECDH (P-256 curve) public/private key pair locally in the browser. The private key is marked non-exportable by the WebCrypto API — it cannot be extracted from the browser context. The public key is displayed to the user as a demonstration of local key material. FYP Part 2 will activate this key pair for full AES-GCM end-to-end message encryption.

7.4 Progressive Web App (PWA)

Reveal-X is fully compliant with the PWA specification. The web app manifest (`manifest.json`) declares the application name, icons, theme colours, and display mode (standalone). A Service Worker script implements a cache-first strategy for static assets (CSS, JavaScript, fonts, icons) and a network-first strategy for API requests, enabling offline access to the chat interface and previously loaded messages.

8. Major Modules

Module	Responsibility	Key Files
Image Preprocessing	Accepts user-selected image; validates format; downscales via Canvas API to target dimensions; converts to greyscale pixel array.	canvas_utils.js
VC Share Generation	Generates cryptographically random Share 1 and derives Share 2 via XOR; encodes Share 1 as Base64 for network transmission.	vc_engine.js
Image Reconstruction	Retrieves Share 1 from received message and Share 2 from local session Map; performs Canvas XOR to reconstruct original image.	vc_engine.js, canvas_utils.js
Real-Time Messaging	Manages WebSocket connections, message routing, typing indicators, read receipts, and presence tracking via Flask-SocketIO.	socket_events.py, socket_client.js
Authentication	Handles user registration, login, session management. Applies script hashing and verifies passwords using Werkzeug.	auth.py, routes.py
Security Layer	Applies DOMPurify sanitisation on all rendered content; server-side HTML escaping in Jinja2; input validation on all endpoints.	app.js, base.html
E2EE Key Module	Generates ECDH P-256 key pair locally using WebCrypto API; displays public key to user; lays groundwork for FYP Part 2 AES-GCM encryption.	crypto.js
PWA / Service Worker	Registers Service Worker; implements cache-first strategy for static assets; network-first for API; manages cache versioning.	service_worker.js, manifest.json
GUI / UI Layer	Glassmorphic dark-mode CSS, responsive breakpoints, micro-animations, ARIA live regions, keyboard navigation, WCAG 2.1 AA compliance.	main.css, glassmorphism.css, animations.css

9. Installation and Setup Guide

9.1 Prerequisites

- Python 3.10 or higher
- pip (Python package manager)
- Node.js (optional — for frontend build tooling)
- Modern web browser: Chrome 90+, Firefox 88+, Safari 15+, or Edge 90+

9.2 Step-by-Step Installation

Step 1 — Clone the Repository

```
git clone https://github.com/your-username/reveal-x.git
cd reveal-x
```

Step 2 — Create a Python Virtual Environment

```
# Create virtual environment
python -m venv venv

# Activate — macOS / Linux
source venv/bin/activate

# Activate — Windows (PowerShell)
.\venv\Scripts\Activate.ps1
```

Step 3 — Install Dependencies

```
pip install -r requirements.txt
```

The requirements.txt file contains:

```
Flask>=2.3.0
Flask-SocketIO>=5.3.6
Werkzeug>=3.0.0
python-socketio>=5.10.0
python-engineio>=4.8.0
eventlet>=0.33.3
```

Step 4 — Configure Environment Variables

Create a .env file in the project root:

```
SECRET_KEY=your-very-secure-secret-key-here
DATABASE_URI=reveal_x.db
DEBUG=False
PORT=5000
HOST=0.0.0.0
```

Step 5 — Run the Application

```
python run.py
```

The application will be accessible at <http://localhost:5000>

Step 6 — Install as PWA (Optional)

Once the application is running in Chrome or Edge, click the install icon in the browser address bar to install Reveal-X as a native-feeling desktop or mobile app. On Safari (iOS/macOS), use Share > Add to Home Screen.

10. Evaluation Plan

10.1 Security Evaluation

The security of the Visual Cryptography implementation will be evaluated by extracting Share 1 from the WebSocket stream and independently analysing its pixel value distribution. A secure share should exhibit a uniform random distribution (entropy ≈ 8 bits per pixel). Any deviation from uniformity indicates a flaw in the random number generation. Reconstructed images from Share 1 alone must produce visually uninterpretable output.

XSS prevention will be evaluated by injecting a standard battery of XSS payloads (OWASP XSS Filter Evasion Cheat Sheet) through all user input vectors: username registration, message text, and any URL parameters. All payloads must be neutralised without breaking legitimate content rendering.

Password hashing will be verified by inspecting stored hash values and confirming the script algorithm identifier and appropriate work factor parameters.

10.2 Image Quality Evaluation

Reconstructed image quality will be quantitatively evaluated using two standard metrics:

- Peak Signal-to-Noise Ratio (PSNR): Measures the ratio between maximum signal power and reconstruction noise. Higher PSNR (measured in dB) indicates closer similarity to the original. A PSNR above 30 dB is generally considered acceptable quality.
- Structural Similarity Index (SSIM): Measures perceptual similarity between the original and reconstructed image across luminance, contrast, and structure dimensions. SSIM ranges from 0 to 1, with 1 indicating perfect similarity.

Baseline PSNR and SSIM values for the classical (2,2) VC reconstruction will be recorded in FYP Part 1. These values will serve as benchmarks against which the AI enhancement module in FYP Part 2 will be evaluated.

10.3 Performance Evaluation

Metric	Measurement Method	Target
Share generation time	<code>browser performance.now()</code>	< 200ms for 1MP image
Reconstruction time	<code>browser performance.now()</code>	< 100ms
WebSocket message latency	timestamp delta sender/receiver	< 50ms on LAN
PSNR (VC reconstruction)	NumPy pixel comparison	> 30 dB baseline
XSS payload neutralisation	OWASP test suite	100% (zero bypass)

11. FYP Part 2 Roadmap

FYP Part 1 establishes the complete foundation: a working visual cryptography engine, real-time messaging, multi-layer security hardening, and a production-grade UI. FYP Part 2 elevates Reveal-X into a fully AI-augmented, end-to-end encrypted communication platform. The four planned development phases are described in detail below.

Phase 1 — ML-Powered Tamper Detection

Research Basis

From the project proposal: 'To detect tampered or manipulated shares using Machine Learning algorithms.' The ML tamper detection module directly fulfils this stated objective.

The Problem: In the current system, if an adversary intercepts Share 1 in transit and modifies even a single region, the receiver reconstructs a corrupted image without any indication of the compromise. There is no share integrity verification mechanism.

The Solution: A dedicated ML tamper detection pipeline integrated at two checkpoints:

- **In-Transit Detection:** A lightweight CNN model will analyse incoming Share 1 packets for pixel value distribution anomalies. Valid VC shares exhibit a statistically uniform pixel distribution. Tampered shares deviate from this distribution in ways detectable by a trained classifier.
- **Pre-Reconstruction Verification:** Before the Canvas API executes the XOR reconstruction, a local model running via ONNX.js in the browser performs a final integrity check on the received share.
- **Alert System:** If tampering is detected, reconstruction is blocked and the user receives a detailed forensic alert identifying the suspected modification region.

Planned Technical Stack: Python (TensorFlow / PyTorch for training), ONNX Runtime for browser-side inference, scikit-learn for statistical baseline models, NumPy for pixel distribution analysis.

Phase 2 — AI Image Enhancement (Deep Learning Super-Resolution)

Research Basis

From the project proposal: 'To improve reconstructed image quality using Deep Learning techniques.' The AI enhancement module directly fulfils this stated objective.

The Problem: Classical 2-of-2 VC reconstruction produces pixel-expanded, noise-degraded output. Reconstructed images exhibit measurably lower PSNR and SSIM compared to the original, particularly for complex images with fine detail or text.

The Solution: A two-stage deep learning post-processing pipeline applied automatically after reconstruction:

- Stage 1 — Denoising CNN: A Convolutional Neural Network trained specifically on visual cryptography reconstruction artefacts. The model learns to suppress the characteristic halftone noise patterns introduced by the XOR share stacking process.
- Stage 2 — AI Upscaling (ESRGAN): An Enhanced Super-Resolution Generative Adversarial Network, fine-tuned on VC-specific training pairs, restores fine detail and approaches original image fidelity. ESRGAN is preferred over bicubic upscaling because it recovers perceptually meaningful high-frequency detail rather than simply smoothing.
- Quality Dashboard: A real-time display of PSNR and SSIM scores before and after enhancement, providing users with quantifiable evidence of quality improvement.

Planned Technical Stack: TensorFlow / PyTorch (model training), ONNX.js (browser-side inference), OpenCV (preprocessing), NumPy (numerical operations).

Phase 3 — Full Peer-to-Peer End-to-End Encryption

The Problem: The ECDH key generation in FYP Part 1 is a UI demonstration. Key exchange and message encryption are not yet end-to-end active over the network.

The Solution: The WebCrypto ECDH key generation is promoted to a fully functional decentralised key-exchange tunnel:

- Automated Key Exchange: On session initiation, ECDH P-256 public keys are exchanged via the WebSocket channel. The shared secret is derived locally by each party using `SubtleCrypto.deriveKey()`.
- AES-GCM Message Encryption: All text messages are encrypted client-side using AES-256-GCM with the ECDH-derived shared key before transmission. The server routes ciphertext it cannot read.
- Forward Secrecy: Session keys are rotated at regular intervals using ephemeral key exchange, ensuring that compromising a long-term key does not expose historical messages.

Phase 4 — Advanced UI/UX Enhancements

- Dynamic Custom Theming: User-selectable colour palettes with live preview, persisted to the user profile and synchronised across devices.
- Advanced Glassmorphism Animations: Procedural background animations (WebGL particle systems, animated gradient meshes) for a more immersive visual environment.
- Forensic Visualisation Panel: An interactive educational panel that animates the visual cryptography process — showing share generation and reconstruction step-by-step.
- WCAG 2.1 AAA Upgrade: High-contrast mode toggle, reduced-motion alternative animations, and enhanced screen reader support.

FYP Part 2 Milestone Timeline

M#	Deliverable	Phase	Target Week
M1	ML Tamper Detection — model training & evaluation	Phase 1	Week 3
M2	ONNX.js in-browser tamper detection integration	Phase 1	Week 5
M3	AI denoising CNN training on VC-specific dataset	Phase 2	Week 7
M4	ESRGAN upscaling integration + PSNR/SSIM dashboard	Phase 2	Week 9
M5	Full ECDH key exchange + AES-GCM message encryption	Phase 3	Week 11
M6	Advanced UI/UX enhancements + WCAG AAA upgrade	Phase 4	Week 13
M7	System integration testing, performance evaluation, final documentation	All	Week 15

12. Roles and Responsibility Matrix

The following Work Breakdown Structure (WBS) defines specific activities, durations, and responsible team members for all project deliverables across both FYP Part 1 and the planned FYP Part 2 scope.

WBS #	Deliverable	Act #	Activity	Duration	Responsible Member(s)
1	Project Planning	1.1	Requirement analysis and problem identification	5 Days	All Members
1	Project Planning	1.2	Literature review on visual cryptography and AI	7 Days	All Members
2	System Design	2.1	Design system architecture and workflow	5 Days	Inam Ullah
2	System Design	2.2	Create diagrams and methodology flow	4 Days	Inam Ullah
3	Image Preprocessing	3.1	Implement image input and preprocessing	6 Days	Inam Ullah
3	Image Preprocessing	3.2	Convert to greyscale and optimise input format	4 Days	Inam Ullah
4	Visual Cryptography Module	4.1	Implement share generation algorithm	7 Days	Inam Ullah
4	Visual Cryptography Module	4.2	Implement image reconstruction mechanism	6 Days	Inam Ullah
5	AI Enhancement Module	5.1	Implement AI-based image enhancement model	8 Days	Muhammad Umar
5	AI Enhancement Module	5.2	Integrate enhancement	5 Days	Muhammad Umar

			with reconstructed image		
6	Tamper Detection Module	6.1	Develop tamper detection logic	6 Days	Muhammad Umar
6	Tamper Detection Module	6.2	Integrate tamper detection with reconstruction process	4 Days	Muhammad Umar
7	GUI Development	7.1	Design graphical user interface	5 Days	Zaka Khan
7	GUI Development	7.2	Integrate GUI with backend system	6 Days	Zaka Khan
8	Testing & Evaluation	8.1	Perform system testing and debugging	7 Days	All Members
8	Testing & Evaluation	8.2	Evaluate image quality and performance metrics	5 Days	All Members
9	Documentation	9.1	Prepare project documentation and report	6 Days	All Members
9	Documentation	9.2	Final review and presentation preparation	4 Days	All Members

13. Conclusion and Future Work

13.1 FYP Part 1 Summary

Reveal-X FYP Part 1 successfully delivers a production-grade, real-time secure messaging web application with a working visual cryptography image sharing engine at its core. The system demonstrates that academic cryptographic research — specifically the Naor-Shamir visual cryptography scheme — can be embedded within a practical, user-facing application without sacrificing usability or accessibility.

The architectural guarantee of server-zero-knowledge image sharing, the multi-layer XSS prevention, script password hashing, and ECDH key generation lay a robust security foundation. The PWA packaging and WCAG 2.1 AA accessibility compliance demonstrate the project's commitment to inclusive, production-ready engineering standards.

13.2 Limitations of the Current Implementation

- The current VC scheme operates on greyscale images only. Colour visual cryptography will be explored in FYP Part 2.
- Reconstructed image quality, while functional, shows measurable PSNR and SSIM degradation relative to the original. AI enhancement (FYP Part 2) will address this.
- The E2EE ECDH key generation is a demonstration only; full AES-GCM message encryption requires the key exchange activation planned for FYP Part 2.
- Tamper detection is absent from the current implementation; the ML-based module is the primary FYP Part 2 deliverable.

13.3 Contribution to Cybersecurity

Reveal-X makes two primary contributions to applied cybersecurity research: first, a practical proof-of-concept demonstrating that information-theoretically secure image sharing is achievable within a real-time web application with acceptable performance; and second, a defined research pipeline for integrating ML-based share integrity verification and AI-based quality restoration with a classical cryptographic scheme.

The project also serves as a foundation for future research in secure multimedia communication, with potential applications in healthcare image transmission, legal evidence sharing, and secure government communications.

13.4 References

- Naor, M., & Shamir, A. (1994). Visual Cryptography. *Advances in Cryptology — EUROCRYPT 1994*. Lecture Notes in Computer Science, vol. 950. Springer, Berlin, Heidelberg.
- Zhou, Z., Arce, G. R., & Di Crescenzo, G. (2006). Halftone Visual Cryptography. *IEEE Transactions on Image Processing*, 15(8), 2441-2453.
- Gonzalez, R. C., & Woods, R. E. *Digital Image Processing* (4th ed.). Pearson Education.
- Goodfellow, I., Bengio, Y., & Courville, A. *Deep Learning*. MIT Press, 2016.

Stallings, W. Cryptography and Network Security: Principles and Practice (8th ed.). Pearson Education.

Wang, X. et al. (2020). Deep Learning Based Image Enhancement for Visual Cryptography Systems. IEEE Access.

OWASP. (2023). XSS Filter Evasion Cheat Sheet. OWASP Foundation.
<https://owasp.org/www-community/xss-filter-evasion-cheatsheet>

14. Approval

Project Supervisor

Comments: _____

Name: _____

Date: _____ Signature: _____

Project Coordinator

Comments: _____

Name: _____

Date: _____ Signature: _____
